Cahier des charges – Gestion du matériel du lycée

Oct 15, 2026 · @lyon

1. Contexte et objectifs

L'application permet au lycée de prêter son matériel (ordinateurs portables, vidéoprojecteurs, caméras, câbles…) en sachant à tout moment qui a quoi, depuis quand, et qui a le droit de faire quoi.

Problème actuel (à confirmer avec le lycée) : les prêts sont suivis sur papier ou dans un tableur. Le matériel se perd, on ne sait pas qui l'a emprunté, et n'importe qui peut modifier le tableur.

Objectifs :

1. Tenir un inventaire à jour du matériel et de son état.
2. Gérer les demandes de prêt, leur validation, la sortie et le retour du matériel.
3. Contrôler strictement les droits : chaque utilisateur ne voit et ne fait que ce que son rôle autorise.
4. Garder une trace de toutes les actions sensibles (qui a fait quoi, quand).
5. Servir de réalisation professionnelle pour l'épreuve E5 du BTS SIO SLAM, avec un volet sécurité mis en avant.

2. Périmètre

La première version se concentre sur le prêt de matériel et les autorisations ; tout le reste est repoussé à plus tard.

Inclus dans la version 1	Exclu (évolutions possibles)
Inventaire du matériel (ajout, modification, archivage)	Achat et commande de matériel
Demandes de prêt, validation, sortie, retour	Application mobile
Gestion des comptes et des rôles	Connexion avec l'ENT du lycée (à étudier)
Journal des actions sensibles	Envoi de SMS
Notifications par e-mail (retards)	Scan de codes-barres / QR codes

Inclus dans la version 1	Exclu (évolutions possibles)
Tableau de bord simple par rôle	Statistiques avancées

3. Acteurs et rôles

L'application compte cinq rôles, du moins puissant au plus puissant ; un utilisateur a un seul rôle à la fois.

Rôle	Qui	Ce qu'il fait en résumé
Visiteur	Personne non connectée	Voit uniquement la page de connexion
Élève	Élèves du lycée	Consulte le catalogue, demande un prêt, suit ses propres prêts
Enseignant	Professeurs	Comme l'élève, plus : demande des prêts pour une classe, valide les demandes des élèves de ses classes
Gestionnaire	Responsable du matériel (technicien, CPE…)	Gère l'inventaire, valide toutes les demandes, enregistre sorties et retours
Administrateur	Service informatique du lycée	Gère les comptes et les rôles, consulte le journal de sécurité, configure l'application

Un administrateur **n'est pas** automatiquement gestionnaire : séparer les deux limite les dégâts si un compte est piraté (principe du moindre privilège, voir section 5).

4. Besoins fonctionnels

Chaque besoin a un identifiant (BF-xx) pour pouvoir le citer dans les tests et dans le dossier E5.

Comptes et connexion

- BF-01 : se connecter avec un identifiant et un mot de passe.
- BF-02 : se déconnecter ; la session expire après 30 minutes d'inactivité.
- BF-03 : réinitialiser son mot de passe par un lien envoyé par e-mail, valable 1 heure.
- BF-04 : l'administrateur crée, modifie, désactive un compte et lui attribue un rôle.

Inventaire

- BF-10 : consulter le catalogue (nom, catégorie, état, disponibilité).
- BF-11 : le gestionnaire ajoute, modifie ou archive un matériel (on n'efface jamais un matériel qui a un historique de prêts).
- BF-12 : le gestionnaire change l'état d'un matériel : disponible, prêté, en panne, en réparation, archivé.

Prêts

- BF-20 : un élève ou un enseignant fait une demande de prêt avec des dates de début et de fin.
- BF-21 : la demande est validée ou refusée (avec un motif obligatoire en cas de refus).
- BF-22 : le gestionnaire enregistre la sortie, puis le retour, avec l'état du matériel au retour.
- BF-23 : un e-mail de rappel est envoyé la veille de la date de retour, puis chaque jour de retard.
- BF-24 : chaque utilisateur voit l'historique de ses prêts ; le gestionnaire voit tous les prêts.

Suivi

- BF-30 : tableau de bord adapté au rôle (mes prêts en cours / demandes à valider / retards).
- BF-31 : l'administrateur consulte et filtre le journal de sécurité (section 6).

5. Gestion des autorisations

Toute action est interdite par défaut ; elle n'est permise que si le rôle de l'utilisateur possède la permission **et** que l'objet visé entre dans son périmètre (ses prêts, ses classes…).

5.1 Deux questions à chaque requête

1. **Authentification** — qui es-tu ? (l'utilisateur est-il connecté, son compte est-il actif ?)
2. **Autorisation** — as-tu le droit ? On vérifie d'abord la permission du rôle, puis le périmètre : par exemple, un élève a la permission de voir un prêt, mais seulement les siens.

5.2 Liste des permissions

Chaque permission a un nom court, utilisé tel quel dans le code et en base de données.

Permission	Description
materiel.voir	Consulter le catalogue
materiel.gerer	Ajouter, modifier, archiver un matériel
pret.demander	Créer une demande de prêt
pret.voir	Consulter des prêts
pret.valider	Accepter ou refuser une demande
pret.annuler	Annuler une demande pas encore sortie
pret.sortie_retour	Enregistrer la sortie et le retour du matériel
compte.gerer	Créer, modifier, désactiver des comptes
role.attribuer	Changer le rôle d'un utilisateur
journal.voir	Consulter le journal de sécurité
parametres.gerer	Modifier les réglages (durée max d'un prêt, etc.)

5.3 Matrice rôles / permissions

« Oui » = sans restriction ; une précision = autorisé seulement dans ce périmètre ; « Non » = interdit. Le visiteur n'a aucune permission.

Permission	Élève	Enseignant	Gestionnaire	Administrateur
materiel.voir	Oui	Oui	Oui	Oui
materiel.gerer	Non	Non	Oui	Non
pret.demander	Oui	Oui	Oui	Non
pret.voir	Les siens	Les siens + élèves de ses classes	Tous	Non
pret.valider	Non	Élèves de ses classes	Tous sauf les siens	Non
pret.annuler	Sa demande en attente	Sa demande en attente	Toutes	Non
pret.sortie_retour	Non	Non	Oui	Non

Permission	Élève	Enseignant	Gestionnaire	Administrateur
compte.gerer	Non	Non	Non	Oui sauf son propre compte
role.attribuer	Non	Non	Non	Oui sauf son propre rôle
journal.voir	Non	Non	Non	Oui
parametres.gerer	Non	Non	Non	Oui

5.4 Règles métier d'autorisation

- **RA-01 Refus par défaut** : une page ou une action sans règle explicite est refusée.
- **RA-02 Contrôle côté serveur** : cacher un bouton ne protège rien ; le serveur revérifie le droit à chaque requête, même si l'utilisateur tape l'URL à la main.
- **RA-03 Contrôle du propriétaire** : avant d'afficher ou modifier un prêt, on vérifie qu'il appartient bien à l'utilisateur (ou à un élève de ses classes). Cela empêche de voir le prêt d'un autre en changeant l'identifiant dans l'URL (faille appelée IDOR).
- **RA-04 Séparation des tâches** : personne ne valide sa propre demande. La demande d'un enseignant ou d'un gestionnaire est validée par un autre gestionnaire.
- **RA-05 Protection des administrateurs** : un administrateur ne peut ni changer son propre rôle ni désactiver son propre compte, et il doit toujours rester au moins un administrateur actif.
- **RA-06 Effet immédiat** : un changement de rôle ou une désactivation ferme les sessions ouvertes de l'utilisateur.
- **RA-07 Limites** : un élève a au maximum N prêts en cours en même temps (N à préciser avec le lycée).
- **RA-08 Refus discret** : un accès refusé affiche une page « Accès refusé » (code HTTP 403) sans détail technique, et la tentative est enregistrée dans le journal.

5.5 Stockage des droits

Les rôles et permissions sont stockés en base, pas écrits en dur dans le code : on peut ainsi ajuster la matrice sans tout réécrire.

- table `roles` (id, nom)
- table `permissions` (id, nom, description)
- table `role_permission` (role_id, permission_id) : quelles permissions a chaque rôle
- colonne `role_id` dans `utilisateurs`

Les restrictions de périmètre (« les siens », « ses classes ») ne sont pas dans ces tables : ce sont des règles écrites dans une fonction dédiée par objet, par exemple `peutVoirPret(utilisateur, pret)`.

Mise en œuvre avec Laravel :

- Le middleware `auth` protège toutes les routes sauf la page de connexion : c'est la réponse à « qui es-tu ? ».
- Un **Gate** par permission vérifie que le rôle de l'utilisateur la possède dans `role_permission` (par exemple `Gate::allows('pret.valider')`).
- Une **Policy** par modèle (`PretPolicy`, `MaterielPolicy`, `UtilisateurPolicy`) contient les règles de périmètre : `view()` vérifie qu'un élève ne voit que ses prêts, `valider()` applique RA-04.
- Dans les contrôleurs, `$this->authorize('valider', $pret)` déclenche la Policy ; si elle répond non, Laravel renvoie automatiquement une erreur 403.
- Dans les vues Blade, `@can('materiel.gerer')` masque les boutons inutiles, pour le confort seulement : la vraie protection reste côté serveur (RA-02).

5.6 Cycle de vie d'un compte

1. **Création** par l'administrateur (un par un, ou import d'un fichier CSV à la rentrée — à préciser), avec le rôle le plus faible adapté.
2. **Première connexion** : mot de passe temporaire, changement obligatoire.
3. **Changement de rôle** par l'administrateur uniquement, enregistré dans le journal (qui, quel ancien rôle, quel nouveau rôle).
4. **Désactivation** en fin d'année ou au départ : plus de connexion possible, mais l'historique des prêts est conservé.
5. **Suppression des données personnelles** après la durée de conservation fixée (voir section 6, RGPD). Un compte ne peut pas être désactivé s'il a encore du matériel non rendu.

5.7 Tests d'autorisation à prévoir

Ces tests prouvent que les règles fonctionnent ; ils sont à garder comme preuves pour l'E5.

Test	Résultat attendu
Visiteur ouvre /prets	Redirigé vers la connexion
Élève A ouvre l'URL du prêt de l'élève B	403 + ligne dans le journal

Test	Résultat attendu
Élève envoie une requête de validation de prêt	403
Enseignant valide la demande d'un élève d'une autre classe	403
Gestionnaire valide sa propre demande	Refusé (RA-04)
Administrateur tente de retirer son propre rôle	Refusé (RA-05)
Rôle d'un élève changé pendant qu'il est connecté	Sa session est fermée
Compte désactivé tente de se connecter	Connexion refusée, message générique

6. Exigences de sécurité

L'application suit les risques de l'OWASP Top 10 ; chaque exigence (ES-xx) dit quelle attaque elle bloque.

Exigence	Protège contre
ES-01 Mots de passe hachés avec `Hash::make() de Laravel` (bcrypt), jamais stockés en clair	Vol de la base de données
ES-02 Mot de passe de 12 caractères minimum	Mots de passe faibles
ES-03 Blocage du compte 15 minutes après 5 échecs de connexion (RateLimiter de Laravel + colonnes echecs_connexion et bloque_jusqua)	Attaque par force brute
ES-04 Message d'erreur identique que l'identifiant ou le mot de passe soit faux	Découverte des comptes existants
ES-05 Accès à la base uniquement par Eloquent ou le Query Builder, qui préparent les requêtes ; pas de SQL écrit à la main avec des variables	Injection SQL
ES-06 Affichage avec `{{ }}` dans Blade (échappement automatique) ; `{!! !!}` interdit sur une donnée saisie par un utilisateur	Faille XSS

Exigence	Protège contre
ES-07 Directive @csrf dans chaque formulaire Blade (Laravel vérifie le jeton automatiquement)	Faille CSRF (action faite à l'insu de l'utilisateur)
ES-08 Nouvel identifiant de session après connexion ; cookie `HttpOnly`, `Secure`, `SameSite`	Vol ou fixation de session
ES-09 HTTPS obligatoire	Interception des échanges
ES-10 Validation de toutes les données reçues avec $request->validate() ou une Form Request (types, longueurs, dates)	Données incohérentes ou malveillantes
ES-11 APP_DEBUG=false dans le fichier .env de production : aucune erreur technique affichée	Fuite d'informations

Journal de sécurité

Sont enregistrés : connexions réussies et échouées, accès refusés (403), changements de rôle, création et désactivation de comptes, validations et refus de prêts. Chaque ligne contient la date et l'heure, l'utilisateur, l'action, l'objet visé et l'adresse IP. Le journal est en lecture seule, même pour l'administrateur.

RGPD

- Données collectées limitées au nécessaire : nom, prénom, classe, e-mail, rôle, historique des prêts.
- Durée de conservation à préciser avec le lycée (par exemple : fin de scolarité + 1 an pour les comptes, 1 an pour le journal).
- Mention d'information visible sur la page de connexion.
- Le responsable de traitement est le lycée ; le contact à indiquer est à préciser.

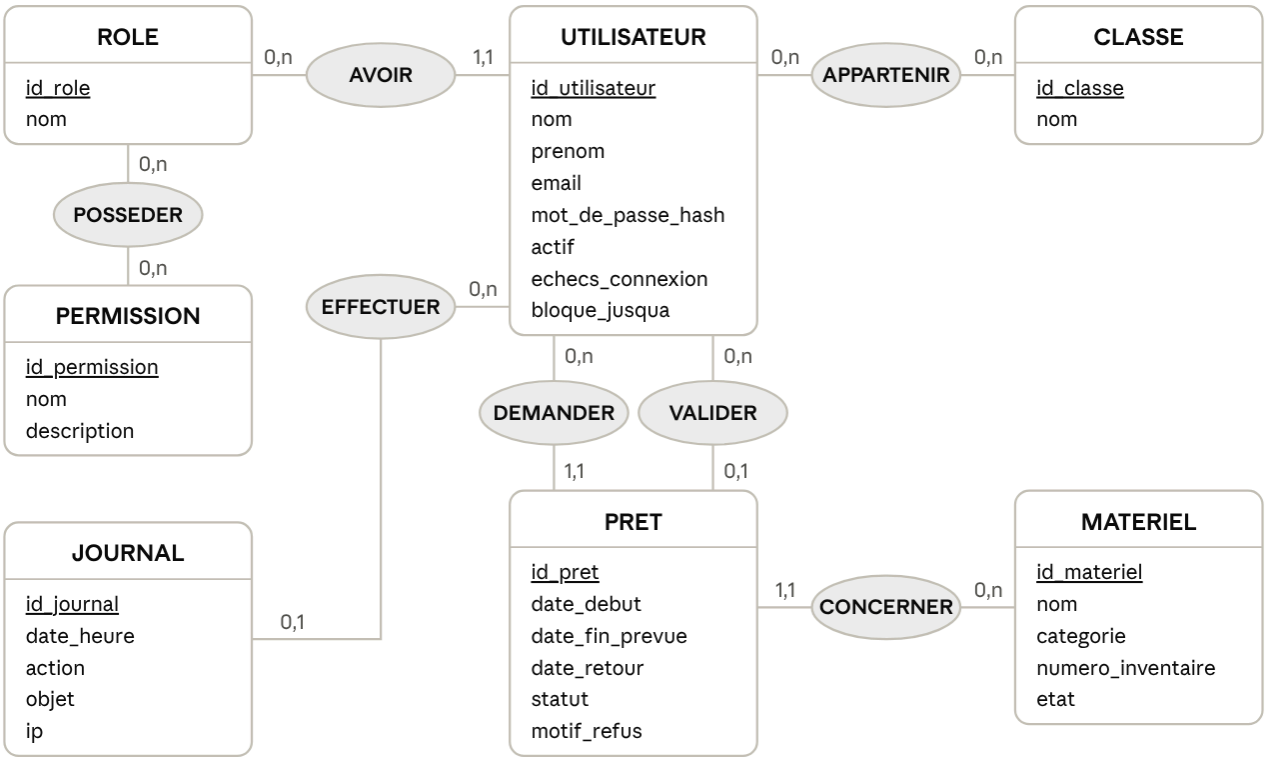
7. Données

Le MCD montre sept entités reliées par sept associations ; il se traduit en neuf tables dans le modèle logique (MLD) ci-dessous.

7.1 Modèle conceptuel de données (MCD)

Un prêt relie un matériel à deux utilisateurs : le demandeur et le validateur

Rectangles = entités (identifiant souligné) · ovales = associations · cardinalités min,max



MCD · 7 entités, 7 associations

Lecture des cardinalités, côté entité : un utilisateur a un et un seul rôle (1,1), un rôle regroupe de 0 à n utilisateurs (0,n). Un prêt est demandé par exactement un utilisateur (1,1) et validé par au plus un (0,1), car il peut être encore en attente. Une ligne du journal a 0 ou 1 utilisateur : une connexion échouée avec un identifiant inconnu n'a pas d'auteur connu.

Association	Règle de gestion	Devient dans le MLD
AVOIR	Un utilisateur a un seul rôle	Clé étrangère role_id dans utilisateurs
POSSEDER	Un rôle a plusieurs permissions, une permission sert à plusieurs rôles	Table role_permission
APPARTENIR	Un élève ou un enseignant appartient à une ou plusieurs classes ; l'admin à aucune	Table classe_utilisateur
DEMANDER	Chaque prêt a un seul demandeur	Clé étrangère demandeur_id dans prets

Association	Règle de gestion	Devient dans le MLD
VALIDER	Un prêt a au plus un validateur, différent du demandeur (RA-04)	Clé étrangère validateur_id (peut être vide) dans prets
CONCERNER	Un prêt porte sur un seul matériel	Clé étrangère materiel_id dans prets
EFFECTUER	Une action du journal a 0 ou 1 auteur	Clé étrangère utilisateur_id (peut être vide) dans journal

Règle de passage : une association (1,1)–(0,n) devient une clé étrangère du côté (1,1) ; une association (0,n)–(0,n) devient une table.

7.2 Modèle logique (tables)

Table	Colonnes principales	Liens
roles	id, nom	—
permissions	id, nom, description	—
role_permission	role_id, permission_id	roles, permissions
utilisateurs	id, nom, prenom, email, mot_de_passe_hash, role_id, actif, echecs_connexion, bloque_jusqua	roles
classes	id, nom (ex. « 1re STMG 2 »)	—
classe_utilisateur	classe_id, utilisateur_id	élèves et enseignants d'une classe
materiels	id, nom, categorie, numero_inventaire, etat	—
prets	id, materiel_id, demandeur_id, validateur_id, date_debut, date_fin_prevue, date_retour, statut, motif_refus	materiels, utilisateurs (×2)
journal	id, date_heure, utilisateur_id, action, objet, ip	utilisateurs

Statuts d'un prêt : `en_attente` → `valide` ou `refuse` → `sorti` → `rendu` (ou `annule` tant qu'il n'est pas sorti). La table `classe_utilisateur` sert à la règle « l'enseignant valide les élèves de ses classes ».

8. Contraintes techniques

Le framework Laravel est imposé ; le reste de la pile reprend ce qui est vu en BTS SIO SLAM, pour que tout soit explicable à l'oral.

Élément	Choix proposé	Statut
Langage serveur	PHP 8.3 minimum (exigé par Laravel 13)	Proposé
Framework	Laravel 13 (version actuelle, sortie en mars 2026), ou la version utilisée en cours	Imposé (version à confirmer avec le prof)
Base de données	MySQL ou MariaDB	Proposé
Serveur web	Apache ou Nginx sur une machine virtuelle Debian	Proposé
HTTPS	Certificat auto-signé en test	Hébergement final à préciser
Versionnement	Git + dépôt GitHub	Proposé
Navigateurs	Versions récentes de Firefox et Chrome, affichage correct sur téléphone	Proposé

Chaque mécanisme Laravel utilisé est expliqué dans la documentation, pour pouvoir le justifier au jury. Les tables gardent leurs noms français : chaque modèle Eloquent l'indique avec la propriété `$table` (par exemple `protected $table = 'prets';`), car par défaut Laravel attend des noms anglais comme `users`.

9. Livrables et critères d'acceptation

Le projet est terminé quand l'application tourne sur la machine virtuelle, que tous les tests de la section 5.7 passent et que la documentation est complète.

Livrables

- [ ] Code source sur GitHub, avec historique de commits clair
- [ ] `README.md` : badges, présentation, capture d'écran, équipe, technologies, installation, licence, lien vers l'architecture
- [ ] `docs/architecture.md` : schéma fonctionnel et schéma technique en Mermaid
- [ ] `docs/autorisations.md` : matrice rôles / permissions et règles RA-01 à RA-08

- [ ] Script SQL de création de la base avec un jeu de données de test (un compte par rôle)
- [ ] Cahier de tests rempli (résultat attendu / obtenu, capture d'écran)
- [ ] Rapport d'audit de sécurité : failles testées, résultat, correction

Critères d'acceptation

- Chaque besoin BF-xx est démontrable en moins de 2 minutes.
- Aucun accès non autorisé possible lors des tests de la section 5.7.
- Aucune faille d'injection SQL ou XSS trouvée avec OWASP ZAP.

10. Planning des deux semaines de congés

Objectif réaliste : finir les comptes, les autorisations et le cycle de prêt de base ; les e-mails et l'audit complet viendront après. Les matinées (environ 3 h) sont pour les révisions de l'examen blanc, les après-midis (environ 3 h) pour le projet.

Jour	Matin : révisions	Après-midi : projet
Lundi S1	Lister les matières et chapitres de l'examen blanc	Relire et compléter ce cahier des charges
Mardi S1	Révisions	Schémas d'architecture + script SQL des 9 tables
Mercredi S1	Révisions	Projet initialisé, connexion / déconnexion (BF-01, BF-02)
Jeudi S1	Révisions	Hachage, blocage après 5 échecs, CSRF (ES-01 à ES-07)
Vendredi S1	Révisions	Rôles et permissions en base + vérification à chaque requête (RA-01, RA-02)
Week-end S1	Repos ou annales	Repos
Lundi S2	Révisions	Gestion des comptes par l'admin (BF-04, RA-05, RA-06)
Mardi S2	Révisions	Inventaire du matériel (BF-10 à BF-12)
Mercredi S2	Révisions	Demandes et validation de prêts (BF-20, BF-21, RA-03, RA-04)
Jeudi S2	Révisions	Journal de sécurité + tests de la section 5.7

Jour	Matin : révisions	Après-midi : projet
Vendredi S2	Examen blanc d'entraînement en conditions réelles	README et `docs/autorisations.md`
Week-end S2	Révisions légères	Repos

Un jour prend du retard ? On décale le projet, jamais les révisions.

11. Instructions pour Claude Code

Cette section transforme le cahier des charges en projet exécutable. Ouvre le dossier du projet dans VS Code, lance Claude Code, et donne-lui les étapes dans l'ordre. Ne lui demande pas de tout faire d'un coup : une étape, tu lis, tu testes, tu passes à la suivante. C'est ce qui te permettra d'expliquer chaque partie ensuite.

Message de départ à donner à Claude Code

Colle d'abord ce cadre, pour qu'il respecte tes règles :

```
Contexte : application Laravel de gestion de prêt de matériel pour un lycée.
Le cahier des charges complet est dans docs/cahier-des-charges.md (colle-le).

Règles de travail :
- Explique chaque fichier que tu crées, en français, en commentaires clairs.
- Pas de code que je ne pourrais pas maintenir seul : pas de package en plus
  de Laravel sans me demander d'abord.
- Avance par petites étapes : une étape, tu t'arrêtes, je teste, je valide.
- Noms des tables et des colonnes en français (cahier des charges, section
7).
- La sécurité des sections 5 et 6 est prioritaire, pas une option.
- À la fin de chaque étape, écris ce que je dois taper pour tester.
```

Étapes, dans l'ordre

Chaque étape correspond à une partie du cahier des charges. Fais-les valider une par une.

1. **Initialisation** : nouveau projet Laravel, base MySQL/MariaDB locale, fichier `.env` (avec `APP_DEBUG=false` prévu pour la production). Vérifie que la page d'accueil s'affiche.
2. **Base de données** : les migrations des 9 tables de la section 7.2, avec les clés étrangères et la table `role_permission`. Un *seeder* crée les rôles, les permissions de la section 5.2, et un compte par rôle pour tester.

3. **Connexion** : authentification avec rôles, hachage des mots de passe, blocage après 5 échecs (ES-01 à ES-04).
4. **Autorisations** : les Gates, les Policies et le middleware de la section 5.5. C'est le cœur du projet : prends le temps de le faire valider.
5. **Inventaire** : les écrans de la section « Inventaire » (BF-10 à BF-12).
6. **Prêts** : demande, validation, sortie, retour (BF-20 à BF-24), avec la règle « personne ne valide sa propre demande » (RA-04).
7. **Journal et tableau de bord** : la journalisation de la section 6 et les écrans par rôle (BF-30, BF-31).
8. **Tests** : les tests d'autorisation de la section 5.7, en tests automatisés.
9. **Documentation** : `README.md` et `docs/architecture.md` (schémas Mermaid), comme en section 9.

Règle de validation entre deux étapes

Ne passe à l'étape suivante que si tu peux répondre oui à ces trois questions sur l'étape en cours :

- Est-ce que ça marche quand je teste à la main ?
- Est-ce que je peux expliquer chaque fichier créé ?
- Est-ce que le cas « accès refusé » renvoie bien une erreur, et pas la page ?